



LOVELY
PROFESSIONAL
UNIVERSITY

**INT332 (DEVOPS VIRTUALIZATION AND
CONFIGURATION MANAGEMENT) COMPUTER
SCIENCE AND ENGINEERING**

Submitted by: Name:

Tarun Rajput

Registration number:

12305028

Roll No.:

35

Section:

40M58

Submitted to :

Mrs. Divya Thakur

Abstract:

This project demonstrates a Dockerized Multi-Tier Web Application using Docker, Docker Compose, AWS EC2, and GitHub Actions. The application consists of a frontend, backend, and MySQL database running inside separate containers. Docker Compose was used for container management and networking, while AWS EC2 and Application Load Balancer were used for cloud deployment. GitHub Actions was integrated to automate the CI/CD workflow and Docker image build process.

Introduction:

Docker is a containerization platform that helps developers package applications along with all their dependencies into lightweight containers. This project demonstrates a Dockerized Multi-Tier Web Application consisting of a frontend, backend, and MySQL database.

The frontend was developed using HTML, CSS, and JavaScript with Nginx, while the backend API was developed using Node.js and Express.js. MySQL was used as the database service. Each component runs inside its own Docker container and all containers are connected using Docker Compose networking.

The project was also deployed on AWS EC2 and integrated with an Application Load Balancer for internet access and traffic distribution. GitHub Actions was used to automate the build process through CI/CD workflow automation.

The main purpose of this project is to understand Docker containerization, multi-tier architecture, cloud deployment, and CI/CD pipeline integration in a practical environment.

Objectives:

The main objective of this project is to understand the implementation of a Multi-Tier Web Application using Docker containerization technology. The project focuses on separating the frontend, backend, and database into different Docker containers and managing them efficiently using Docker Compose.

Another objective of this project is to learn how containerized applications can be deployed on cloud platforms such as AWS EC2. The project also aims to provide practical knowledge of container networking, port mapping, load balancing, and CI/CD automation using GitHub Actions.

This project helps in understanding modern DevOps practices and cloud deployment techniques used in real-world applications.

Technologies Used:

The following technologies and tools were used in this project:

- **HTML** – Used to create the structure of the frontend web pages.

- **CSS** – Used for styling and designing the frontend interface.
- **JavaScript** – Used to add functionality and interaction in the frontend.
- **Node.js** – Used for developing the backend server.
- **Express.js** – Used as a backend framework for creating REST APIs.
- **MySQL** – Used as the database for storing application data.
- **Docker** – Used to containerize the frontend, backend, and database services.
- **Docker Compose** – Used to manage and connect multiple containers together.
- **Nginx** – Used as a web server for serving the frontend application.
- **AWS EC2** – Used for deploying and hosting the application on the cloud.
- **Application Load Balancer (ALB)** – Used to distribute incoming traffic across services.
- **GitHub Actions** – Used to automate the CI/CD pipeline and Docker image build process.

Project Architecture:

The project follows a Multi-Tier Architecture where the application is divided into three different layers: Frontend, Backend, and Database. Each layer runs inside its own Docker container and communicates with other containers through Docker Compose networking.

Frontend Layer

The frontend layer was developed using HTML, CSS, and JavaScript and served through the Nginx web server. It provides the user interface and sends requests to the backend API.

Backend Layer

The backend layer was developed using Node.js and Express.js. It handles API requests from the frontend and processes application logic. The backend also communicates with the MySQL database for data operations.

Database Layer

The database layer uses MySQL to store and manage application data. It runs inside a separate Docker container and is connected to the backend container through Docker networking.

Docker Compose Integration

Docker Compose was used to manage all containers together. It automatically creates a common network between the frontend, backend, and database containers, allowing them to communicate with each other.

AWS Deployment Architecture

The complete application was deployed on an AWS EC2 Ubuntu instance. An Application Load Balancer (ALB) was configured to distribute incoming traffic and provide internet access to the frontend application.

Project Workflow:

The project workflow begins with the development of the frontend and backend application. The frontend was created using HTML, CSS, and JavaScript, while the backend API was developed using Node.js and Express.js. MySQL was used as the database service for storing application data.

Separate Dockerfiles were created for the frontend and backend services to containerize the applications. Docker Compose was then used to configure and manage all the containers together, including the frontend, backend, and MySQL database containers.

When the project is executed using Docker Compose, all containers start automatically and communicate through Docker networking. The frontend sends API requests to the backend, and the backend processes the requests and interacts with the MySQL database.

After successful local execution, the complete application was deployed on an AWS EC2 Ubuntu instance. Security groups and inbound rules were configured to allow internet access to the application.

An AWS Application Load Balancer (ALB) was also configured to distribute incoming traffic and provide reliable access to the frontend application. Finally, GitHub Actions CI/CD workflow was integrated to automate the Docker image build process whenever code changes are pushed to the GitHub repository.

Docker Implementation:

Docker was used to containerize all components of the Multi-Tier Application. Separate Docker containers were created for the frontend, backend, and MySQL database services. This approach ensures application portability, consistency, and easier deployment across different environments.

A separate Dockerfile was created for the frontend application, where Nginx was used as the web server to serve static files such as HTML, CSS, and JavaScript. Another Dockerfile was created for the backend application developed using Node.js and Express.js.

Docker Compose was used to manage all the containers together using a single `docker-compose.yml` file. It helped in defining container services, networking, port mapping, environment variables, and dependencies between services.

The frontend container was mapped to port 3010, the backend container to port 5000, and the MySQL database container to port 3309. Docker Compose automatically created a shared network that allowed communication between all containers.

The application was executed using Docker Compose commands, which simplified the process of building, starting, stopping, and managing the complete multi-container application.

GitHub Actions CI/CD:

GitHub Actions was used to implement Continuous Integration and Continuous Deployment (CI/CD) automation for the project. A workflow file named `docker.yml` was created inside the `.github/workflows` directory.

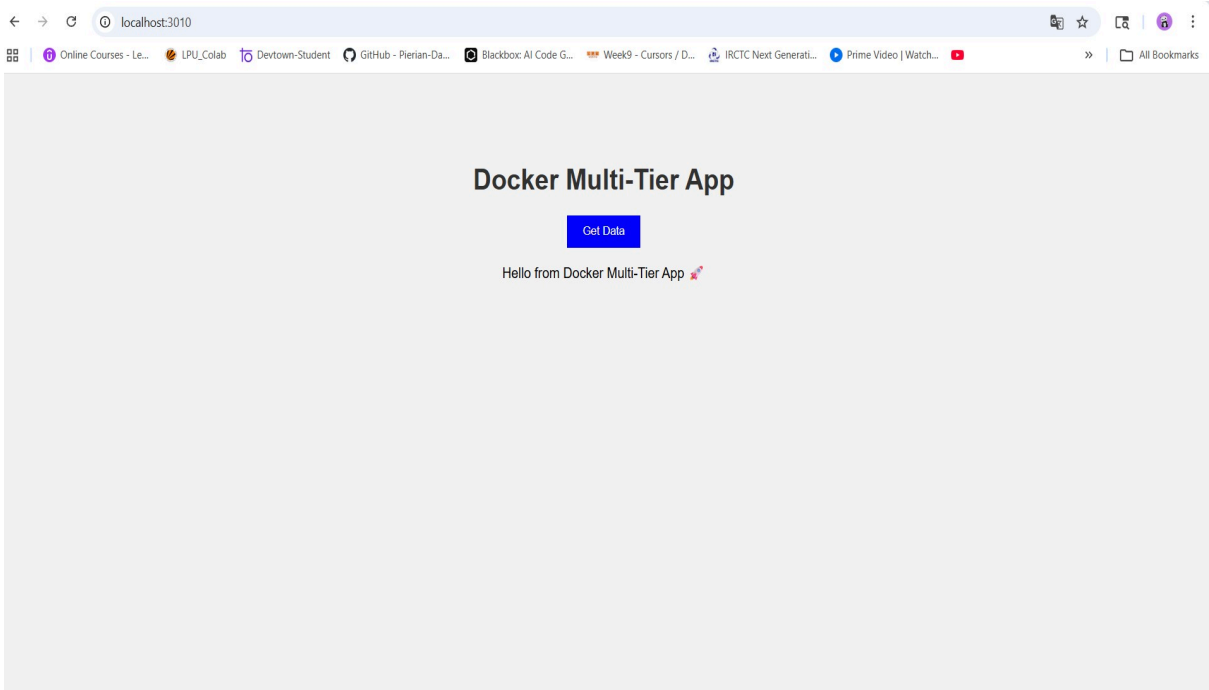
The CI/CD pipeline automatically triggers whenever code changes are pushed to the GitHub repository. The workflow performs tasks such as checking out the repository code, building Docker images for the frontend and backend services, and verifying that the application builds successfully.

This automation reduces manual effort and helps maintain consistency during the development and deployment process. GitHub Actions also helps in identifying build issues early by automatically running the workflow after every push to the repository.

The integration of CI/CD in this project demonstrates modern DevOps practices used in real-world software development environments.

Github link: <https://github.com/TarunUk/Docker-Multi-Tier-Application>

Output Screen Shots:



```
File Edit View

version: "3.8"

services:

  frontend:
    build: ./frontend
    ports:
      - "3010:80"

  backend:
    build: ./backend
    ports:
      - "5000:5000"
    depends_on:
      - mysql

  mysql:
    image: mysql:8
    container_name: mysql-compose
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: multitierdb
    ports:
      - "3309:3306"
```

Containers

Images

Volumes

Kubernetes

Builds

Models

MCP Toolkit

Docker Hub

Docker Scout

Extensions

Containers

Container CPU usage

Container memory usage

0.88% / 1200% (12 CPUs available)

500.79MB / 7.29GB

Show charts

Search

Only show running containers

Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
------	--------------	-------	---------	---------	--------------	---------

Terminal

```
=> => exporting attestation manifest sha256:49835715a3f2071baf69b1e60c898ffe67af3a6ae65838d5fb87d19ec8490e
=> => exporting manifest list sha256:c388399087fdccfd060e4566f990c47ddbc36c4bes2715d1c91fb875ca41e44
=> => naming to docker.io/library/docker-multi-tier-application-frontend:latest
=> => unpacking to docker.io/library/docker-multi-tier-application-frontend:latest
=> [frontend] resolving provenance for metadata file
[+] Running 6/6
✔ docker-multi-tier-application-frontend Built 0.0s
✔ docker-multi-tier-application-backend Built 0.0s
✔ Network docker-multi-tier-application_default Created 0.2s
✔ Container docker-multi-tier-application-frontend-1 Started 2.4s
✔ Container mysql-compose Started 2.3s
✔ Container docker-multi-tier-application-backend-1 Started 2.6s
PS C:\docker-multi\docker-multi-tier-application> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
f94f3b6d75ee   docker-multi-tier-application-backend "docker-entrypoint.s..." About a minute ago Up About a minute 0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
543a53aed247   docker-multi-tier-application-frontend "/docker-entrypoint..." About a minute ago Up About a minute 0.0.0.0:3010->80/tcp, [::]:3010->80/tcp
7d9b09b7ac88   mysql:8                             "docker-entrypoint.s..." About a minute ago Up About a minute 0.0.0.0:3309->3306/tcp, [::]:3309->3306/tcp
PS C:\docker-multi\docker-multi-tier-application>
```

Windows Pow...

Engine running RAM 1.98 GB CPU 0.17% Disk: 19.62 GB used (limit 1006.85 GB)

Docker Multi-Tier App

Get Data

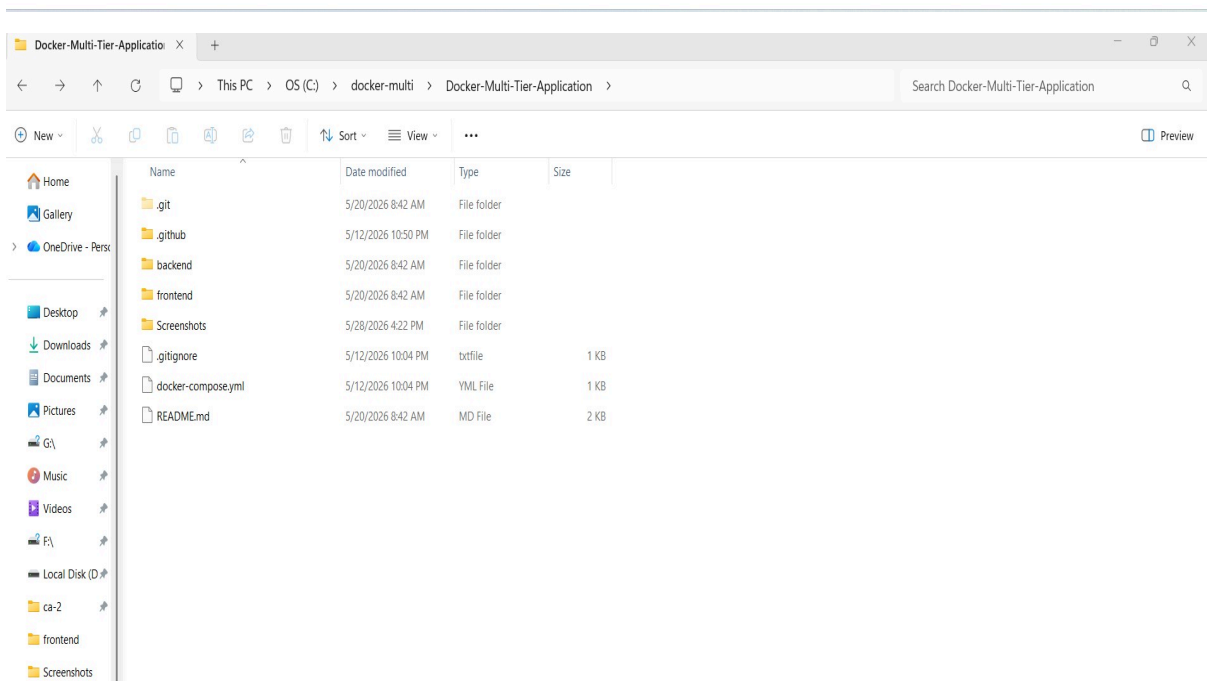


Docker Multi-Tier Application

This project demonstrates containerization using Docker and Docker Compose.

Get Backend Data

Hello from Dockerized Node.js Backend 🚀



EC2

Instances

Dashboard

AWS Global View

Events

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Capacity Manager

Images

AMIs

AMI Catalog

Elastic Block Store

Instances (1/2) Info

Last updated less than a minute ago

Connect

Instance state

Actions

Launch instances

Saved filter sets

Choose filter set

Find Instance by attribute or tag (case-sensitive)

	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS
☐	myserver	i-0798e012c1a085f14	Running	t3.micro	3/3 checks passed	us-east-1a	-
☒	MyDocker	i-0e0451be1fe17bfb8	Running	t3.micro	3/3 checks passed	us-east-1a	-

i-0e0451be1fe17bfb8 (MyDocker)

Details

Status and alarms

Monitoring

Security

Networking

Storage

Tags

Instance summary Info

Instance ID

i-0e0451be1fe17bfb8

Public IPv4 address

-

Private IPv4 addresses

10.0.2.108

Instance state

Running

Public DNS

-

IPv6 address

-

Private IP DNS name (IPv4 only)

ip-10-0-2-108.ec2.internal

Hostname type

IP name: ip-10-0-2-108.ec2.internal

CloudShellFeedbackConsole Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. PrivacyTermsCookie preferences

aws

Search

[Alt+S]

Ask Amazon

United States (N. Virginia)

Tarun Rajput (982189530346)

Tarun Rajput

```
ubuntu@ip-10-0-1-50:~$ git clone https://github.com/TarunUK/Docker-Multi-Tier-Application.git
Cloning into 'Docker-Multi-Tier-Application'...
remote: Enumerating objects: 37, done.
remote: Counting objects: 100% (37/37), done.
remote: Compressing objects: 100% (31/31), done.
remote: Total 37 (delta 8), reused 30 (delta 3), pack-reused 0 (from 0)
Receiving objects: 100% (37/37), 546.83 KiB | 26.04 MiB/s, done.
Resolving deltas: 100% (8/8), done.
ubuntu@ip-10-0-1-50:~$ cd Docker-Multi-Tier-Application
ubuntu@ip-10-0-1-50:~/Docker-Multi-Tier-Application$ sudo docker compose up -d --build
WARNING[0000] /home/ubuntu/Docker-Multi-Tier-Application/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 11/13
  ⠋ mysql [252.2MB / 253.2MB Pulling]
  ⠋ r148eeb1a828 Download complete
  ⠋ 63b1a26b5169 Download complete
  ⠋ ab54944e00e7 Download complete
  ⠋ ca90c44920b4 Download complete
  ⠋ 1dced004aaec Extracting 1 s
  ⠋ b9f6fc0eaa0c Download complete
  ⠋ d3bae02b6aad Download complete
  ⠋ 63882f528d32 Downloading [132.4MB/132.4MB]
  ⠋ 7865e9f32c41 Download complete
  ⠋ 765b9b45fc65 Download complete
  ⠋ 95cf9b00eaf5 Download complete
  ⠋ eb33bcfc4506 Download complete
```

i-0286e5cb4d78baefa (docker-multi-tier)

PublicIPs: 100.53.251.99 PrivateIPs: 10.0.1.50

CloudShellFeedbackConsole Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. PrivacyTermsCookie preferences

TarunUK / Docker-Multi-Tier-Application

Search

Code

Issues

Pull requests

Actions

Projects

Wiki

Security and quality

Insights

Settings

Actions

New workflow

All workflows

Management

Caches

Attestations

Runners

Usage metrics

Performance metrics

All workflows

Showing runs from all workflows

Filter workflow runs

Help us improve GitHub Actions

Tell us how to make GitHub Actions work better for you with three quick questions.

Give feedback

6 workflow runs

Workflow

Event

Status

Branch

Actor

updated frontend and backend

Docker CI #6: Commit 5bdc1b4 pushed by TarunUK

main

May 20, 8:42 AM GMT+5:30

30s

Update README.md

Docker CI #5: Commit 675b969 pushed by TarunUK

main

May 18, 3:23 PM GMT+5:30

34s

deployed on aws ec2 server

Docker CI #4: Commit f180c32 pushed by TarunUK

main

May 16, 12:27 AM GMT+5:30

34s

Update README.md

Docker CI #3: Commit 7b85073 pushed by TarunUK

main

May 14, 9:41 PM GMT+5:30

33s

trigger github actions and edited readme

May 14, 10:54 PM GMT+5:30

Conclusion:

This project successfully demonstrated the implementation of a Dockerized Multi-Tier Web Application using modern DevOps and cloud technologies. The frontend, backend, and MySQL database were containerized using Docker and managed efficiently using Docker Compose.

The project provided practical knowledge of containerization, container networking, port mapping, and multi-container application management. Deployment on AWS EC2 and integration with an Application Load Balancer helped in understanding cloud hosting and traffic management concepts.

Additionally, the implementation of GitHub Actions CI/CD workflow automated the build process and introduced real-world DevOps practices. Overall, this project helped in gaining hands-on experience with Docker, cloud deployment, and CI/CD automation techniques used in modern software development environments.

